

파이썬 기초 프로그래밍과 응용

250219 - 3일차

경로 최적화를 위한 알고리즘

- BFS(Breadth-First search)
- DFS(Depth-First Search)
- 다익스트라 (Dijkstra) 알고리즘
- A* 알고리즘

계산 복잡도 (Computational Complexity)

알고리즘이 문제를 해결하는 데 필요한 자원(시간과 공간)의 양을 측정하여 나타내는 방법

- 시간 복잡도(Time Complexity): 알고리즘이 실행되는 데 걸리는 시간.
- 공간 복잡도(Space Complexity): 알고리즘이 사용하는 메모리의 양.

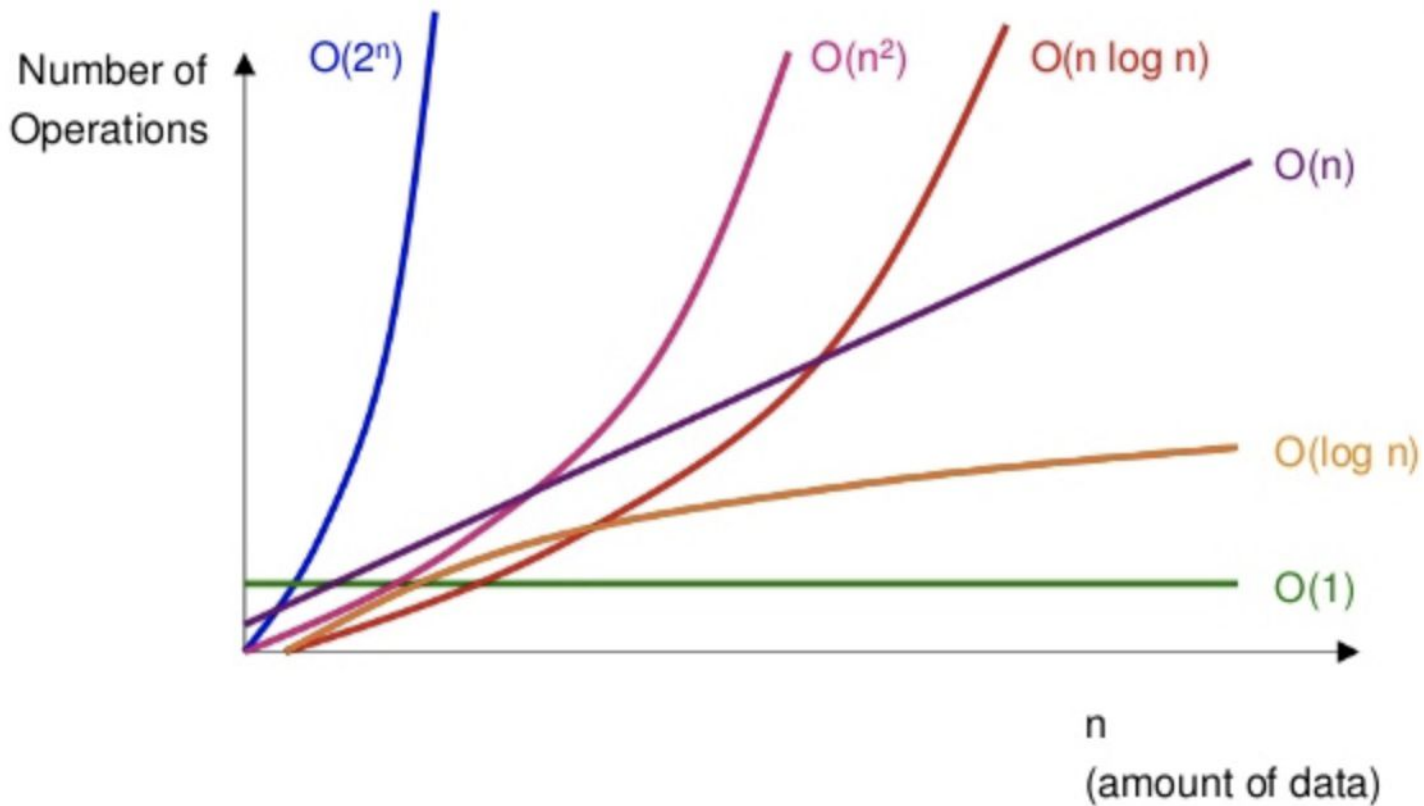
계산 복잡도

입력 크기 n 에 따라 증가하는 알고리즘의 실행 시간을 근사적으로 나타냄

점근 표기법(Asymptotic Notation)

- 빅오 표기법(O): 최악의 경우를 나타냄.
- 빅오메가 표기법(Ω): 최선의 경우를 나타냄.
- 빅세타 표기법(Θ): 평균적인 경우를 나타냄.

계산 복잡도



계산 복잡도

이진 로그: 밑이 2인 $\log$

- $\log_2(x)$: 일반적인 표기
- $\text{ld}(x)$: 국제 표준
- $\text{lg}(x)$: 미국 논문에서 자주 등장

계산 복잡도 예시

$O(n^n)$: 모든 가능한 순열을 생성하는 알고리즘

$O(2^n)$: 피보나치 수열을 재귀로 계산하는 알고리즘

$O(n^2)$: 단순 정렬 알고리즘

$O(n \log n)$: 퀵 정렬(Quick Sort) <https://youtu.be/kPRA0W1kECg>

$O(n)$: 최대값 찾기, 모든 수의 합 계산하기

$O(1)$: 특정 위치의 값을 읽기

계산 복잡도

그래프의 입력 요소

V: 그래프의 정점(Vertex) 개수

E: 그래프의 간선(Edge) 개수

BFS와 DFS의 계산복잡도

인접 리스트의 경우: $O(V+E)$

인접 행렬의 경우: $O(V^2)$

계산 복잡도



```
graph_dict = {  
    1: [2, 3],  
    2: [4, 5],  
    3: [6]  
}
```

```
graph_matrix = [  
    [0, 1, 1, 0, 0, 0],  
    [0, 0, 0, 1, 1, 0],  
    [0, 0, 0, 0, 0, 1],  
    [0, 0, 0, 0, 0, 0],  
    [0, 0, 0, 0, 0, 0],  
    [0, 0, 0, 0, 0, 0]  
]
```

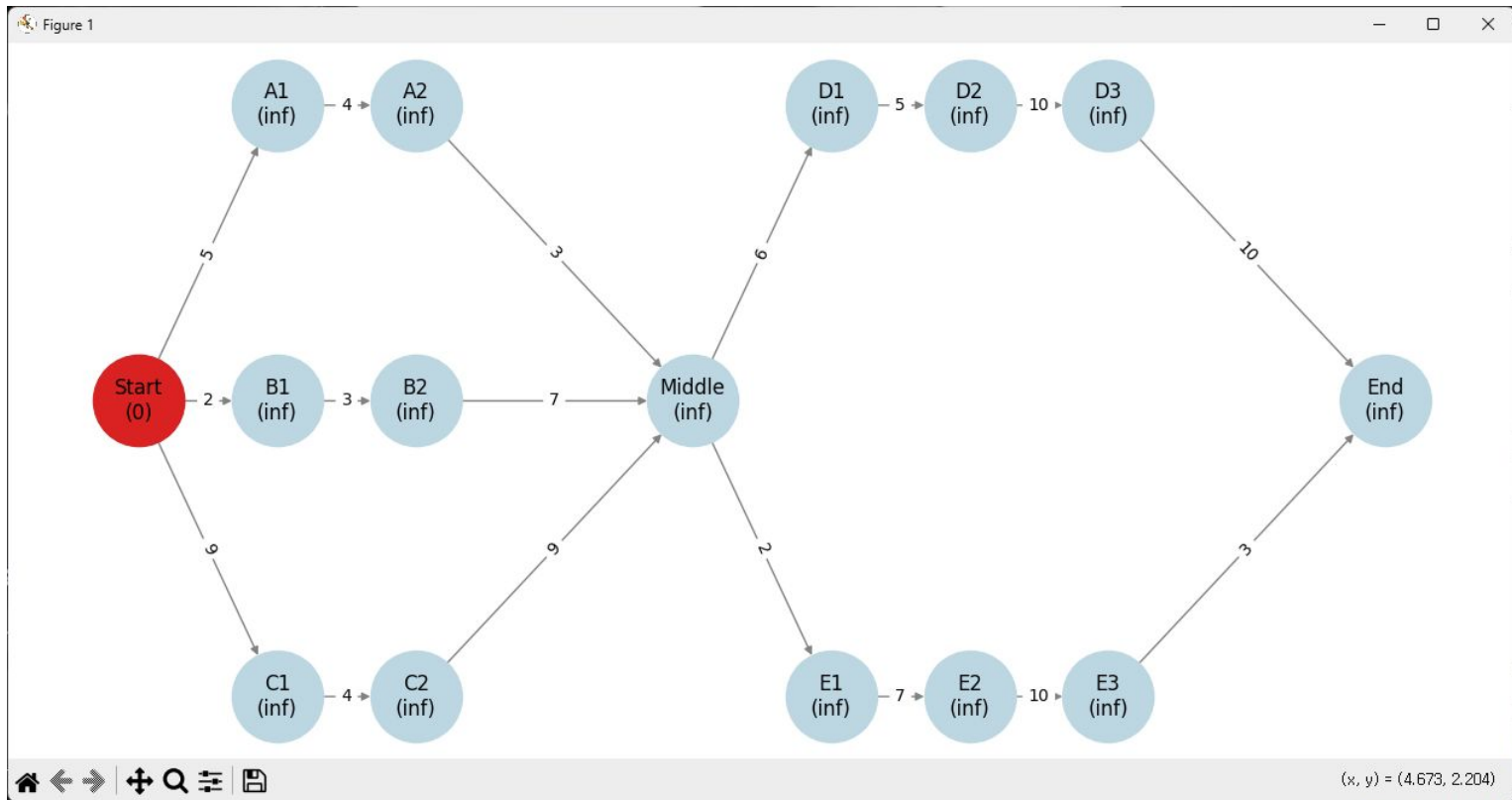
다익스트라(Dijkstra) 알고리즘

- 한 노드에서 다른 모든 노드까지의 최단 경로를 찾는 알고리즘
- 그리디 알고리즘
 - 매 단계에서 가장 비용이 적은 경로를 선택
- 음의 가중치가 없는 그래프에서만 사용가능

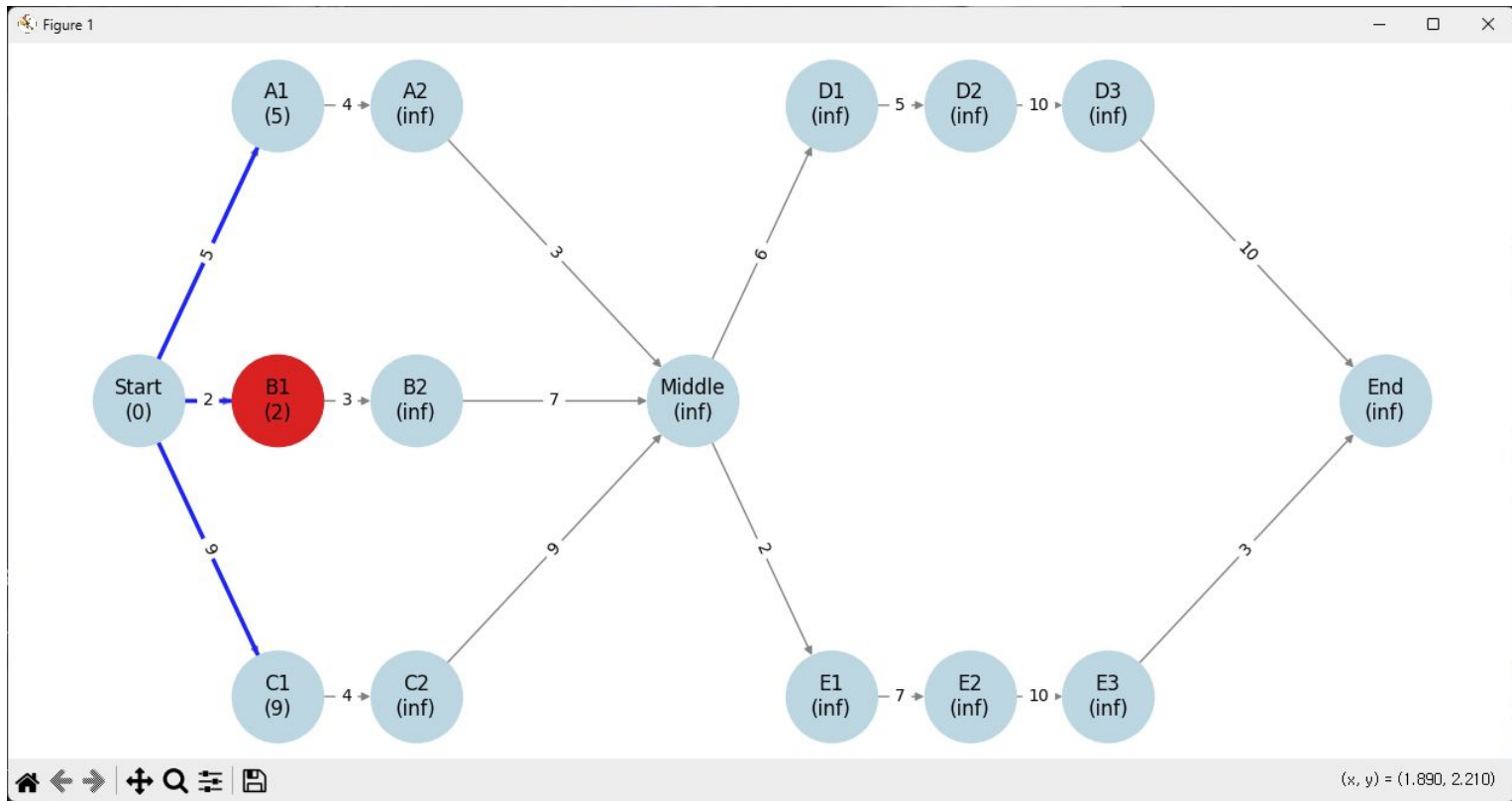
다익스트라(Dijkstra) 알고리즘

1. 시작 노드의 거리를 0으로 설정하고, 나머지 노드의 거리는 무한대로 초기화합니다.
2. 방문 여부를 확인하기 위한 집합을 생성합니다.
3. 방문하지 않은 노드 중 가장 짧은 거리를 가진 노드를 선택합니다.
4. 선택된 노드를 기준으로 인접한 노드들의 거리를 계산하고, 더 작은 값으로 갱신합니다.
5. 해당 노드를 방문 처리합니다. 3번으로 돌아가 반복합니다.
6. 모든 노드를 방문하거나 더 이상 갱신할 수 없는 경우 알고리즘이 종료됩니다.

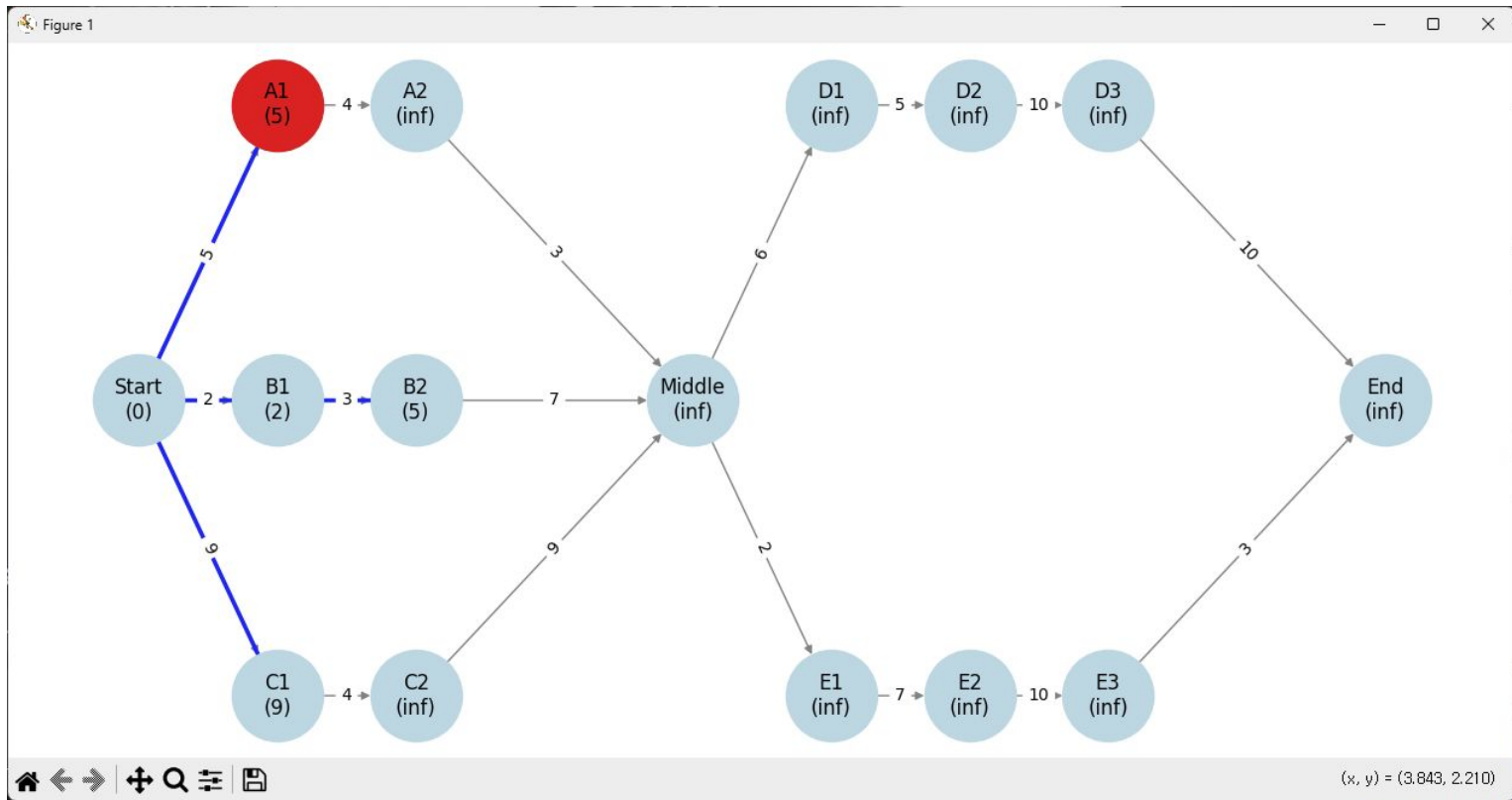
다익스트라(Dijkstra) 알고리즘



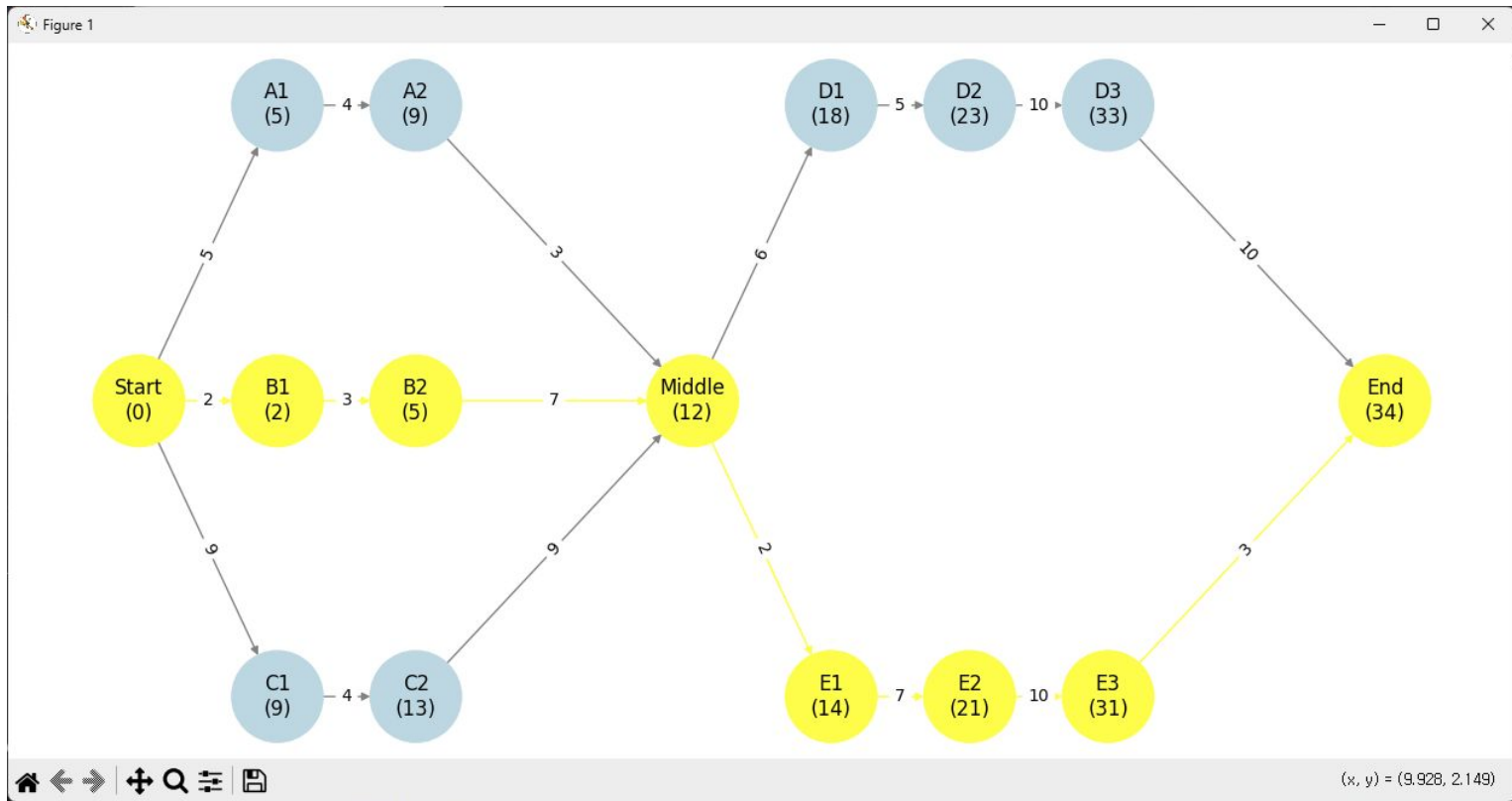
다익스트라(Dijkstra) 알고리즘



다익스트라(Dijkstra) 알고리즘



다익스트라(Dijkstra) 알고리즘



다익스트라(Dijkstra) 알고리즘

시간 복잡도

- 단순 구현 시 $O(V^2)$
- 인접 리스트와 힙을 사용하면 $O((V+E)\log V)$

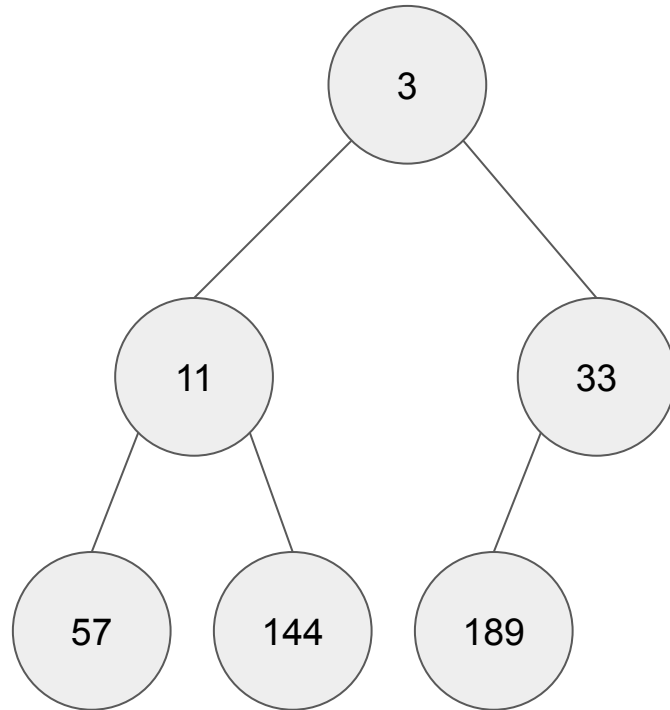
힙 (Heap)

완전 이진 트리(Complete Binary Tree)를 기반으로 한 구조

부모 노드와 자식 노드 간의 특정 정렬 조건(힙 속성)을 만족해야 합니다.

- 최대 힙(Max Heap): 부모 노드의 값이 자식 노드보다 크거나 같음.
- 최소 힙(Min Heap): 부모 노드의 값이 자식 노드보다 작거나 같음.

힙 (Heap)

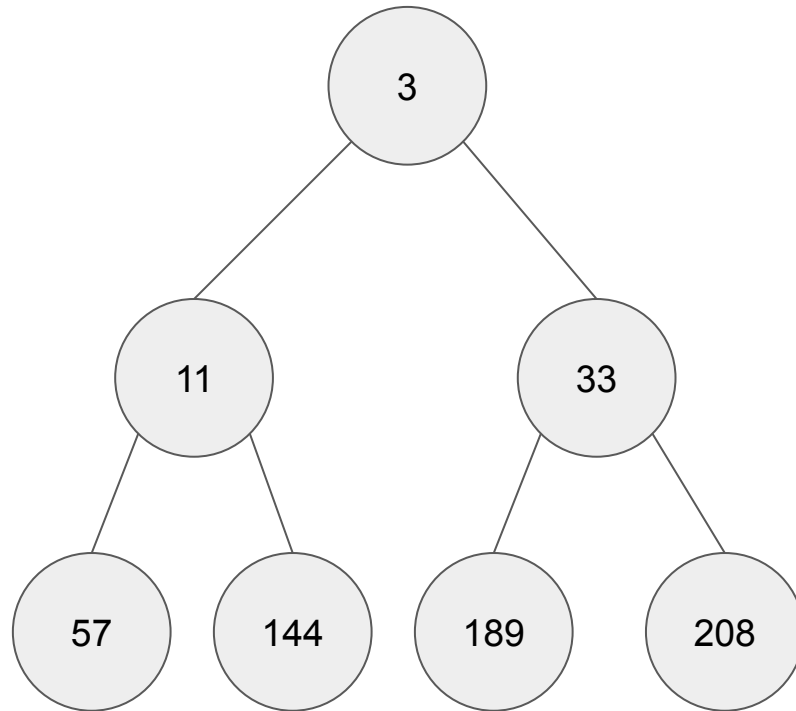


힙 (Heap)

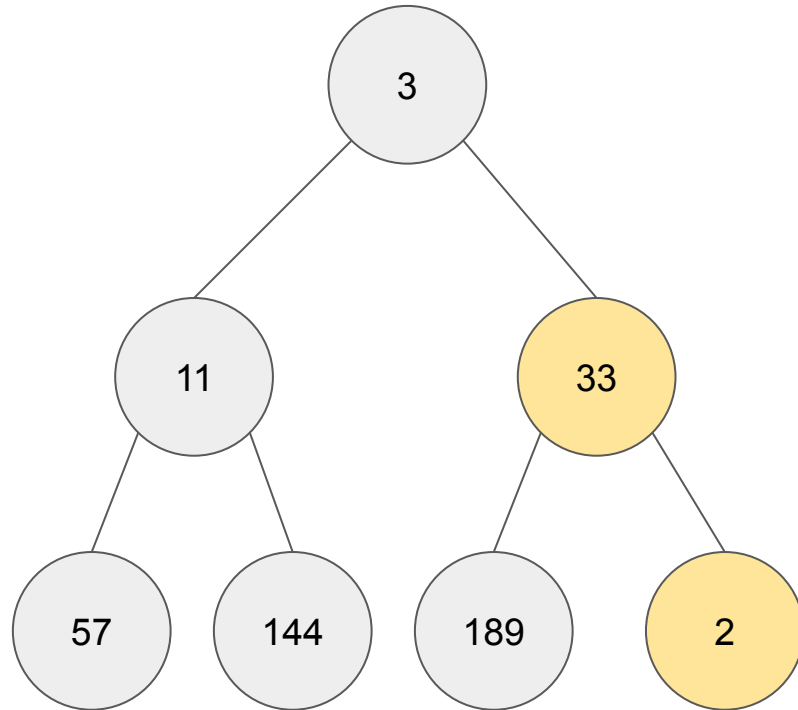
연산 시간 복잡도

- 최댓값(최솟값) 조회: $O(1)$
- 삽입 및 삭제(히피파이, Heapify): $O(\log N)$

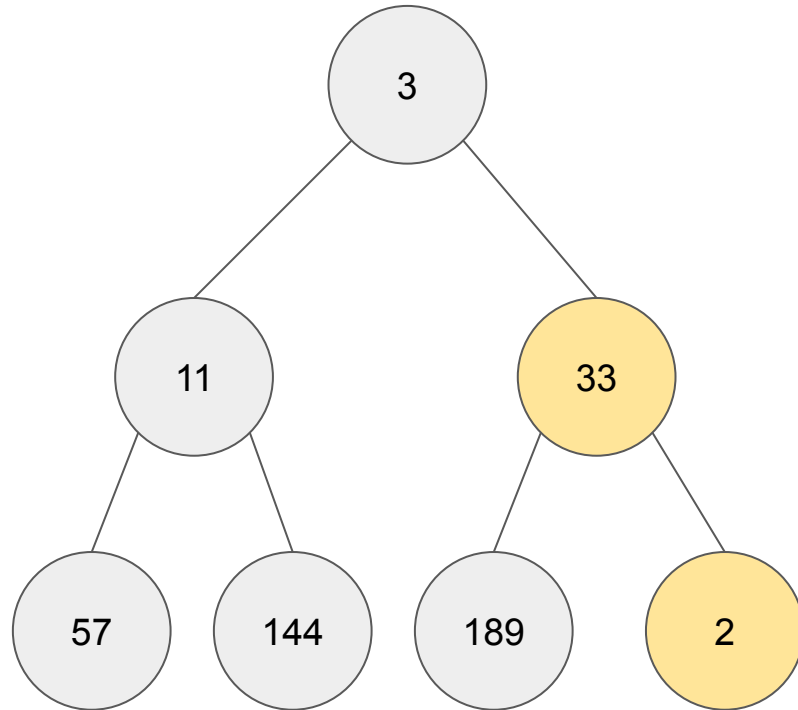
힙 (Heap)



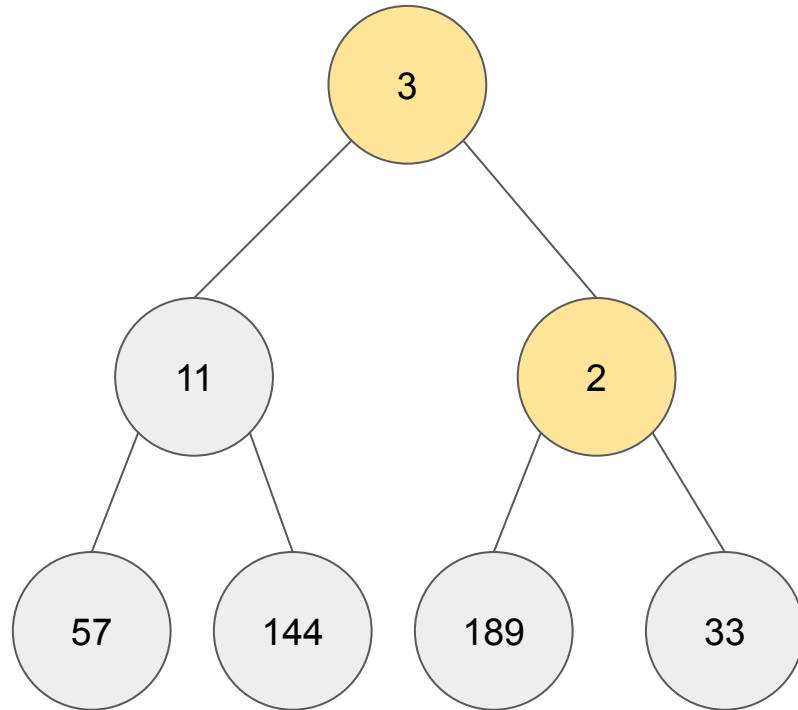
힙 (Heap)



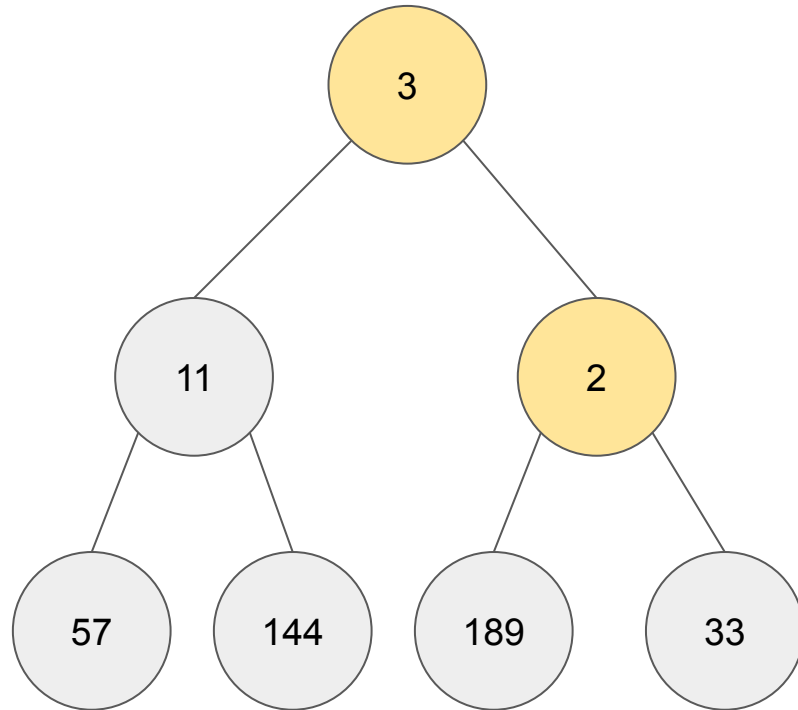
힙 (Heap)



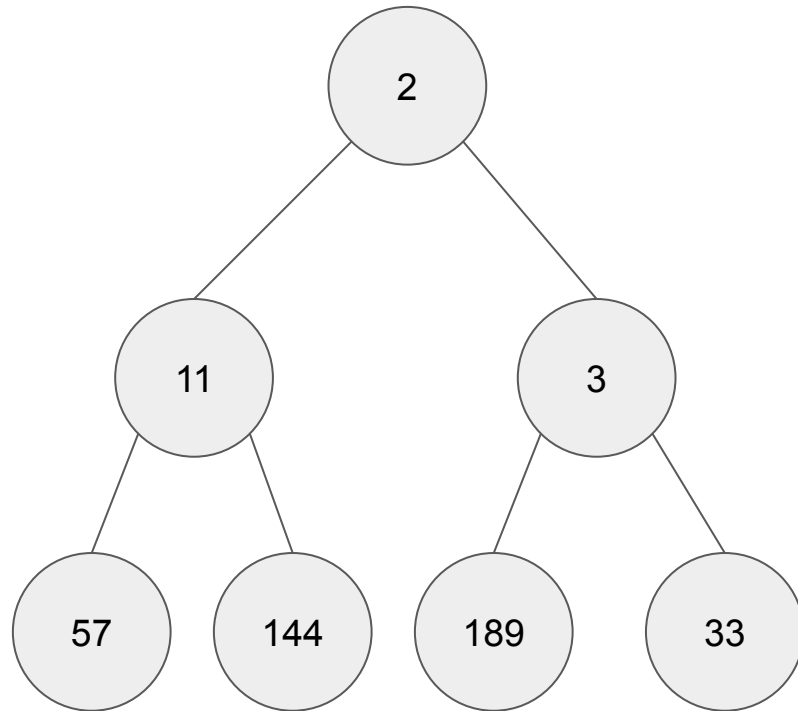
힙 (Heap)



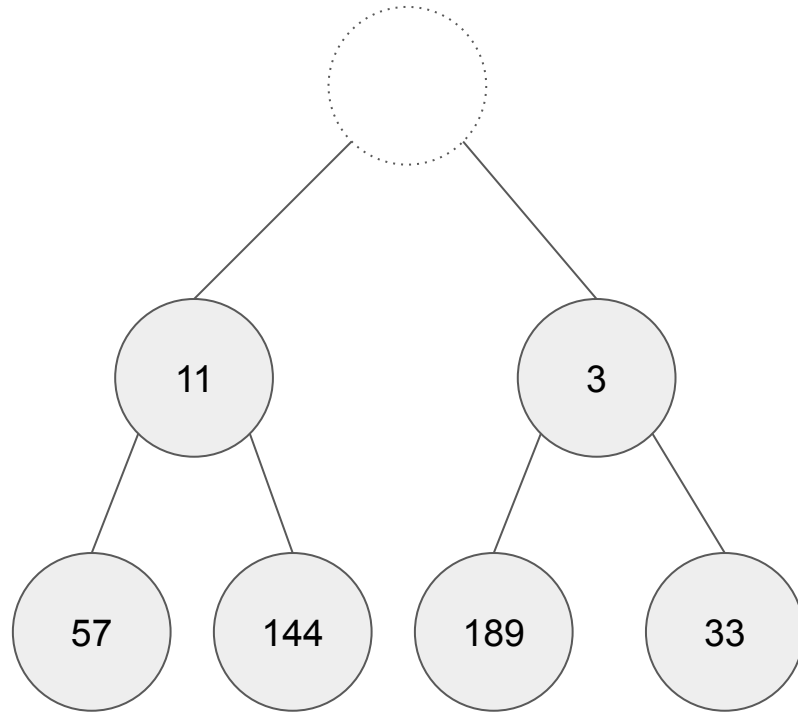
힙 (Heap)



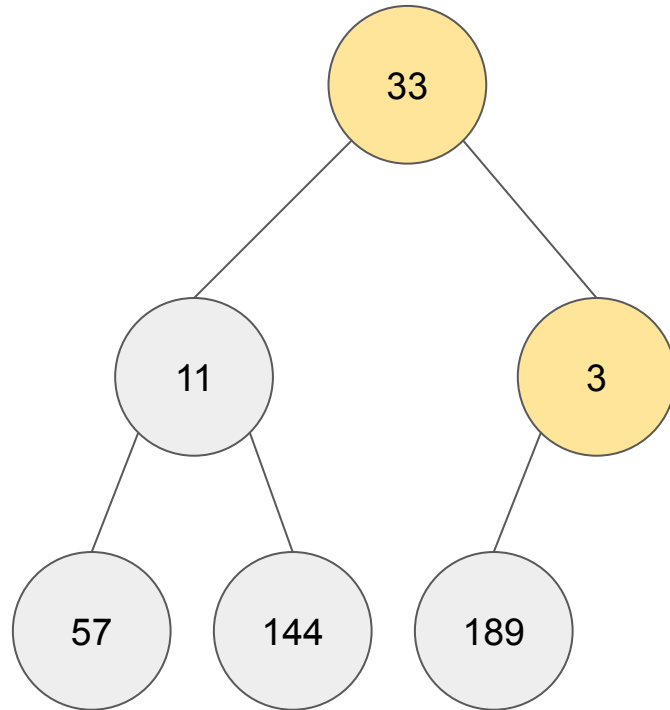
힙 (Heap)



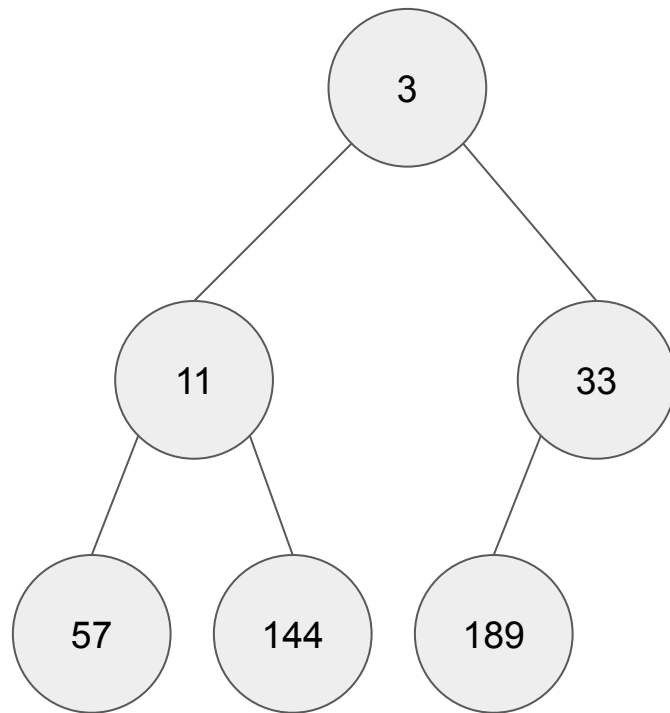
힙 (Heap)



힙 (Heap)



힙 (Heap)



최적화된 다익스트라 알고리즘과 비교

```
(.venv) PS D:\marker-main> python .\13_complexity_compare.py
```

Test with 500 nodes and 24726 edges:

Time without heap: 0.0138 seconds

Time with heap: 0.0051 seconds

Speedup: 2.70x

Test with 1000 nodes and 100331 edges:

Time without heap: 0.0507 seconds

Time with heap: 0.0188 seconds

Speedup: 2.70x

Test with 5000 nodes and 2503555 edges:

Time without heap: 1.3395 seconds

Time with heap: 0.4359 seconds

Speedup: 3.07x

Test with 10000 nodes and 9994310 edges:

Time without heap: 5.2978 seconds

Time with heap: 1.7447 seconds

Speedup: 3.04x

Test with 20000 nodes and 19990235 edges:

Time without heap: 16.1276 seconds

Time with heap: 3.5604 seconds

Speedup: 4.53x

Test with 5000 nodes and 250392 edges:

Time without heap: 0.6874 seconds

Time with heap: 0.0569 seconds

Speedup: 12.07x

Test with 5000 nodes and 2498965 edges:

Time without heap: 1.3392 seconds

Time with heap: 0.4561 seconds

Speedup: 2.94x

Test with 5000 nodes and 14996010 edges:

Time without heap: 4.5679 seconds

Time with heap: 2.1411 seconds

Speedup: 2.13x

A* (A-Star) 알고리즘

- 다익스트라 알고리즘과 유사
- 휴리스틱(heuristic)을 사용하여 탐색 속도를 높임

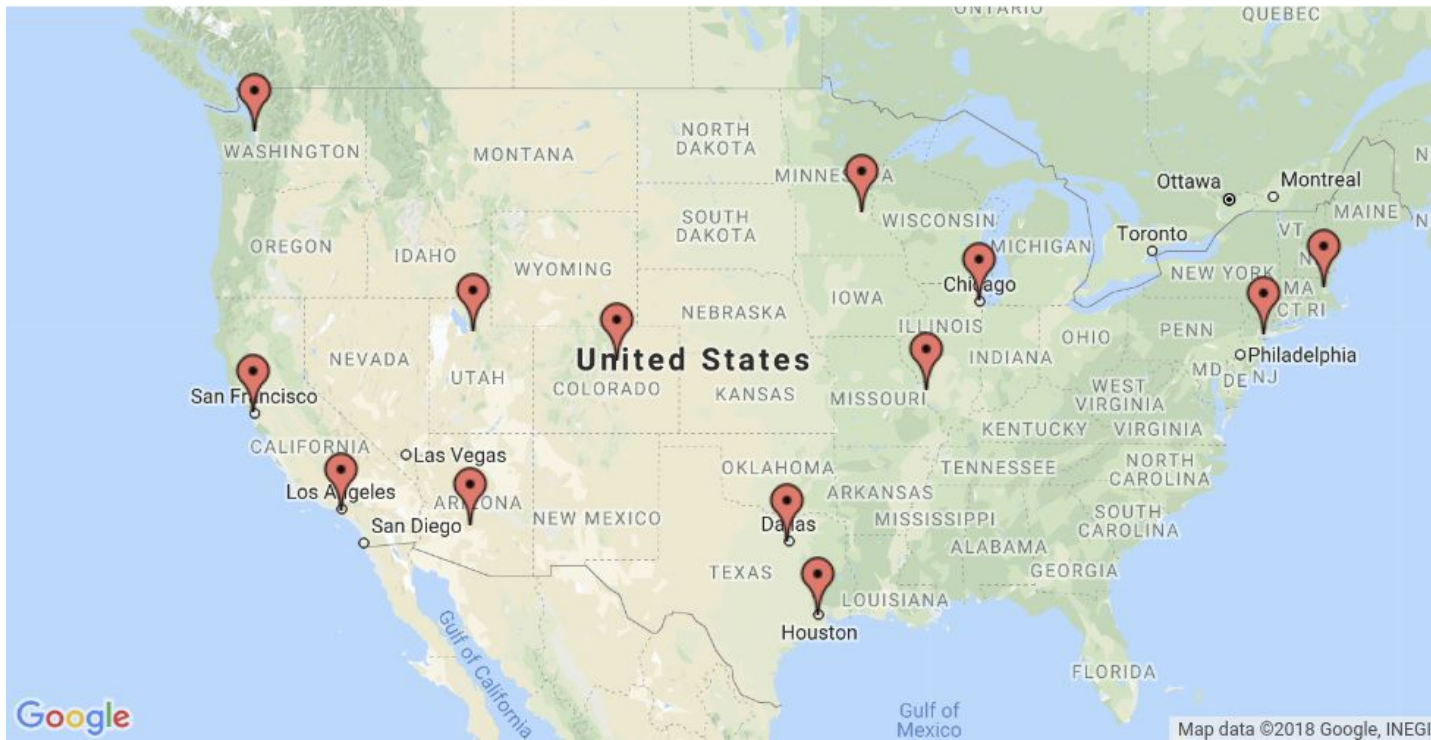
A* (A-Star) 알고리즘

- 휴리스틱 가중치 계산: $f(n) = g(n) + h(n)$
 - $g(n)$: 시작 노드에서 현재 노드까지의 실제 비용 (경로 가중치)
 - $h(n)$: 현재 노드에서 목표 노드까지의 추정 비용 (휴리스틱 값)

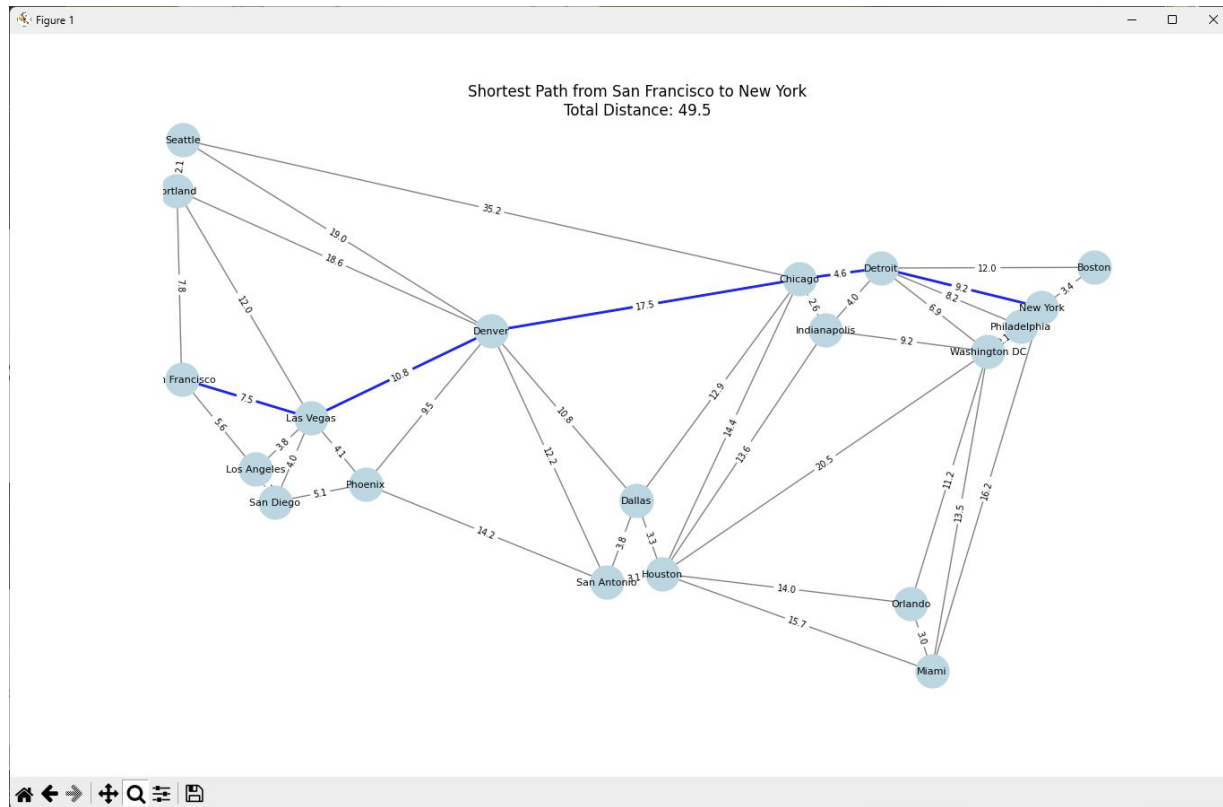
A* (A-Star) 알고리즘

1. 시작 노드가 들어있는 열린 목록(Open List)과 빈 닫힌 목록(Closed List)을 만들어 초기화
2. 열린 목록에서 $f(n)$ 이 가장 작은 노드를 선택
3. 선택한 노드와 인접한 노드들을 탐색
 - a. 선택한 노드가 목표 노드라면 탐색 종료하고 최단 경로를 반환
4. 인접 노드가
 - a. 닫힌 목록에 있는 경우, 열린 목록에 없으면 추가합니다.
 - b. 열린 목록에 있는 경우, 더 낮은 $g(n)$ 값을 가진 경로가 발견되면 해당 경로를 업데이트합니다.
5. 2번으로 돌아가서 반복

A* (A-Star) 알고리즘



A* (A-Star) 알고리즘



다익스트라 알고리즘과 비교

```
• (.venv) PS D:\marker-main> python .\coad.py
```

```
Test with 2000 nodes and 400164 edges:
```

```
Time with A*: 6.5642 milliseconds
```

```
Time with Dijkstra: 71.0530 milliseconds
```

```
Time Speedup: 10.82x
```

```
A* Path Length: 65.0642560703561
```

```
Dijkstra Path Length: 42.48313379213165
```

```
Path Length Ratio (A*/Dijkstra): 1.53x
```

```
Test with 5000 nodes and 2498739 edges:
```

```
Time with A*: 15.5304 milliseconds
```

```
Time with Dijkstra: 449.9539 milliseconds
```

```
Time Speedup: 28.97x
```

```
A* Path Length: 39.6909715823637
```

```
Dijkstra Path Length: 27.545779759554552
```

```
Path Length Ratio (A*/Dijkstra): 1.44x
```

```
Test with 10000 nodes and 5000009 edges:
```

```
Time with A*: 43.4030 milliseconds
```

```
Time with Dijkstra: 973.1299 milliseconds
```

```
Time Speedup: 22.42x
```

```
A* Path Length: 114.57091871158457
```

```
Dijkstra Path Length: 104.89277893267578
```

```
Path Length Ratio (A*/Dijkstra): 1.09x
```

```
Test with 20000 nodes and 19998711 edges:
```

```
Time with A*: 128.7454 milliseconds
```

```
Time with Dijkstra: 3964.0650 milliseconds
```

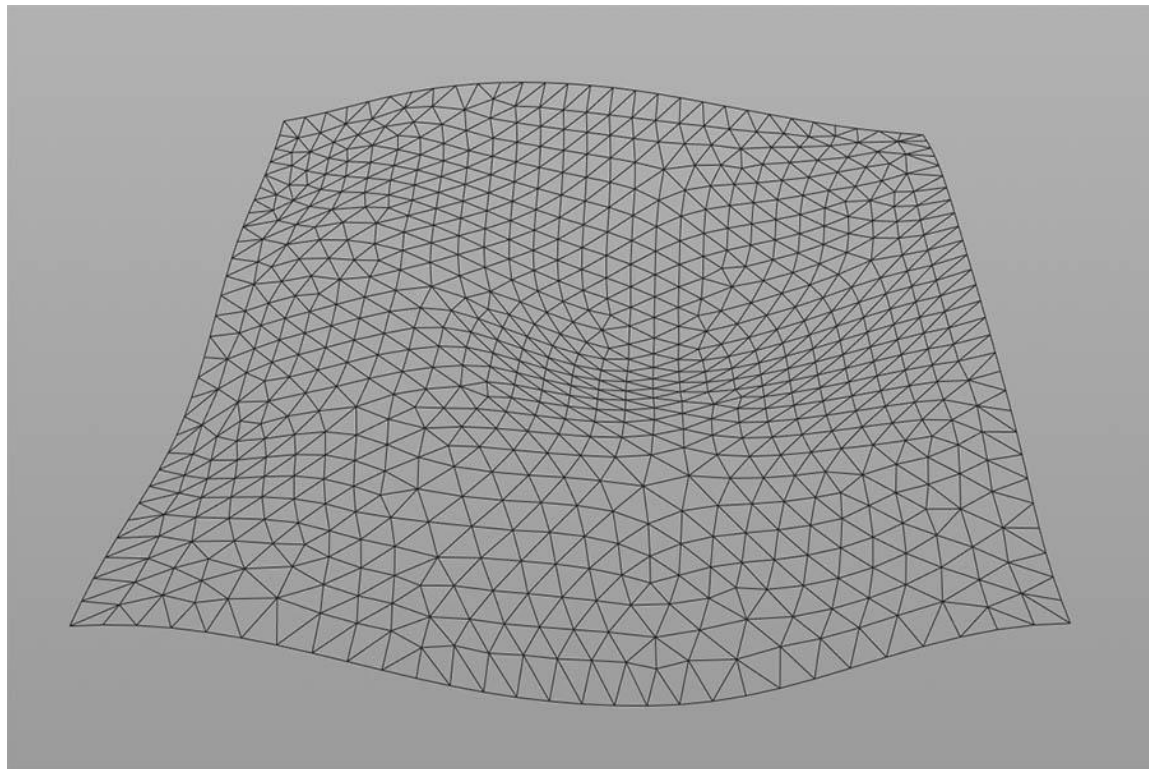
```
Time Speedup: 30.79x
```

```
A* Path Length: 83.2286794714535
```

```
Dijkstra Path Length: 76.47918245401246
```

```
Path Length Ratio (A*/Dijkstra): 1.09x
```

A* (A-Star) 휴리스틱 함수 커스텀하기



A* (A-Star) 휴리스틱 함수 커스텀하기

• `(.venv) PS D:\Github\marker> python .\practice_5_astar_optimization.py`

algorithm	duration (s)	path length
manhattan	4.4163	1015
weighted manhattan	4.97939	1019
euclidean	4.16325	1015
chebyshev	4.36197	1005
your_heuristic	4.42497	1015

외판원 순회 문제 (Traveling Salesman Problem, TSP)

모든 도시를 한 번씩 방문하고 돌아오는 경로를 찾는 문제

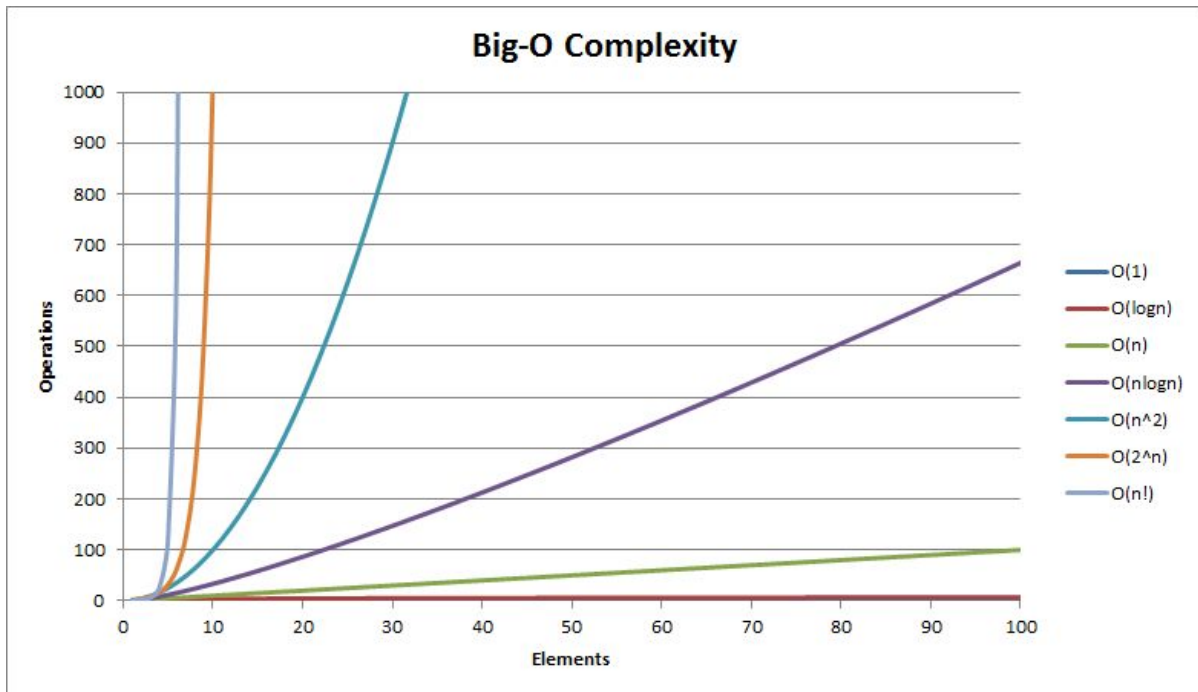
외판원 순회 문제는 자체는 $O(n!)$ 의 복잡도를 가짐

- 최저의 비용이 드는 경로를 찾는 것은 대표적인 **NP-Hard** 문제
- 특정 이하의 비용이 드는 경로를 찾는 것은 대표적인 **NP-Complete** 문제

문제 클래스

문제가 쉽다? 어렵다?

- 계산 복잡도 기준
- 다항시간 = $O(n^k)$,
 $k \neq n$
- 지수시간 = $O(k^n)$



문제 클래스

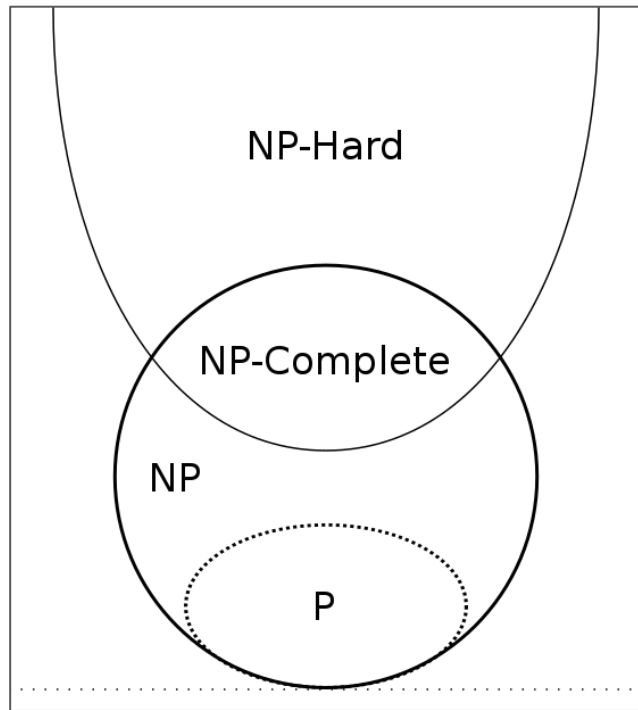
P 문제: 해답을 찾기 쉬운 결정 문제

NP 문제: 해답을 검증하기 쉬운 결정 문제

NP-Hard 문제: 해답을 찾기 힘든 문제

NP-Complete 문제:

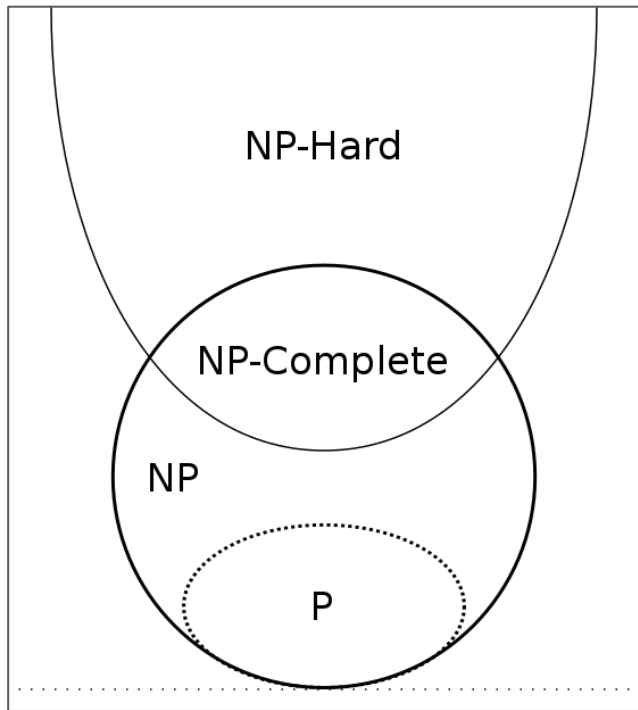
- 해답을 찾기 힘든 결정 문제인데
- 해답을 쉽게 검증할 수 있는 결정 문제



문제 클래스

결정 문제?

- 예 또는 아니오로 답할 수 있는 문제
 - 예 일때는 그 해답을 반환
 - 아니오 일때는 해답을 반환하지 않음
- **5 x 4 은 15 입니까?**
 - 아니오.
- 루빅스 큐브를 원래 상태로 되돌릴 수 있는 방법이 있습니까?
 - 네, 다음과 같은 방법으로 되돌릴 수 있습니다.



100만 달러

- $P=NP$ 문제: "모든 NP 문제가 P에 속하는가?"
 - 밀레니엄 문제
 - 네/아니오 둘 중 하나임을 증명할 수 있으면 100만달러
-
- 전문가 99%는 $P \neq NP$ 일 것이다 라고 추측
 - 하지만 증명되지 않음

외판원 순회 문제 (Traveling Salesman Problem, TSP)

이 경로는 모든 도시를 한 번씩 돌면서 시작도시로 돌아오는 최저비용 경로인가요?

=> 결정 문제

=> 제시한 경로가 최저비용인지 쉽게 확신할 수 없음

=> NP-Hard 문제

외판원 순회 문제 (Traveling Salesman Problem, TSP)

모든 도시를 한 번씩 돌면서 시작도시로 돌아오는
경로 중에서 비용이 k 이하인 경로가 있나요?

=> 결정 문제

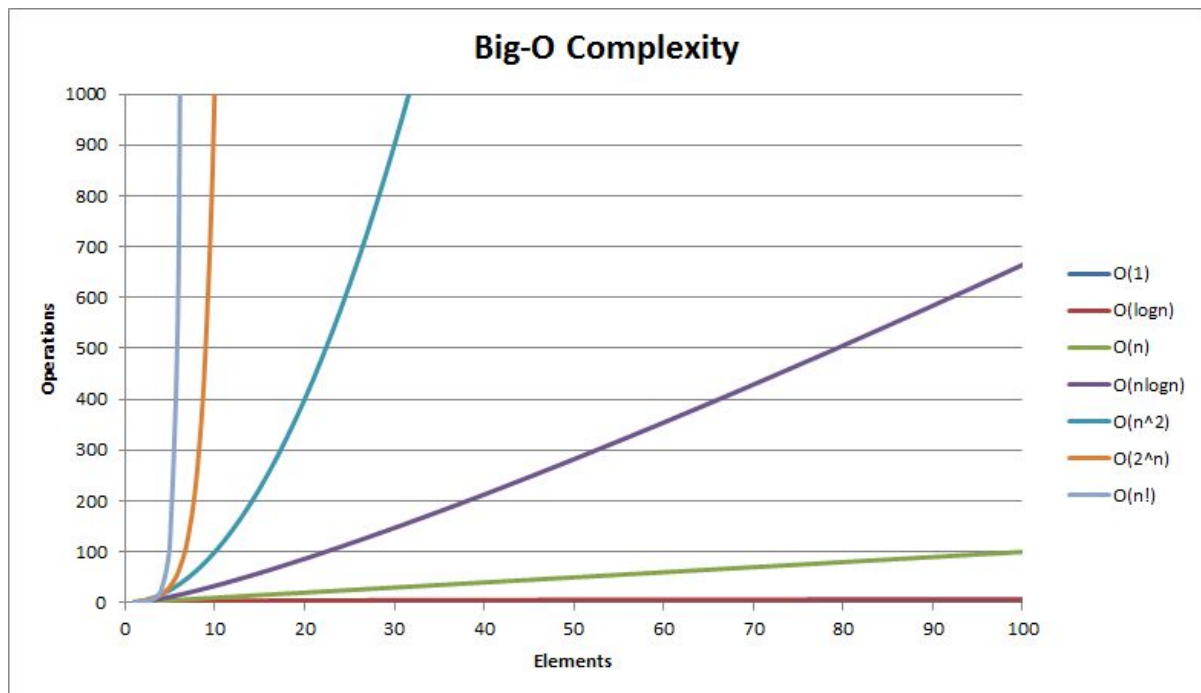
=> 네/아니오 라고 답하기 위해서는 모든 경로 $O(n!)$ 를 검증

=> 대답이 '네' 일때는 준 경로의 합을 k 와 비교

=> NP-Complete 문제

외판원 순회 문제 (Traveling Salesman Problem, TSP)

- NP-Complete를 포함하는 NP-Hard 문제는 합리적인 시간내에 풀 수 없음



외판원 순회 문제 (Traveling Salesman Problem, TSP)

- 모든 경우의 수 검토 (Brute-Force)
- 휴리스틱 사용
 - Nearest Neighbor (NN) 알고리즘
 - 2-opt 알고리즘

외판원 순회 문제 (Traveling Salesman Problem, TSP)

- Nearest Neighbor (NN) 알고리즘
 - a. 임의의 시작 도시에서 출발
 - b. 현재 도시에서 가장 가까운 미방문 도시를 다음 방문지로 선택
 - c. 모든 도시를 방문할 때까지 2번 과정을 반복
 - d. 마지막 도시에서 시작 도시로 돌아와 순회를 완성

외판원 순회 문제 (Traveling Salesman Problem, TSP)

- Nearest Neighbor (NN) 알고리즘

- 장점

- 구현이 매우 간단
 - 실행 속도가 빠름 (시간복잡도: $O(n^2)$)

- 단점

- 최적해를 찾지 못하는 경우가 많음
 - 지역적 최적해(Local Optimum)에 빠지기 쉬움

외판원 순회 문제 (Traveling Salesman Problem, TSP)

- 2-opt 알고리즘: 기존 경로를 조금씩 개선해나가는 알고리즘
 - a. NN 알고리즘 등으로 초기 해를 생성
 - b. 두 개의 간선을 선택하여 제거
 - c. 새로운 방식으로 경로를 재연결
 - d. 새 경로가 더 짧으면 교체
 - e. 더 이상 개선이 없을 때까지 b-e 과정을 반복

외판원 순회 문제 (Traveling Salesman Problem, TSP)

- 2-opt 알고리즘

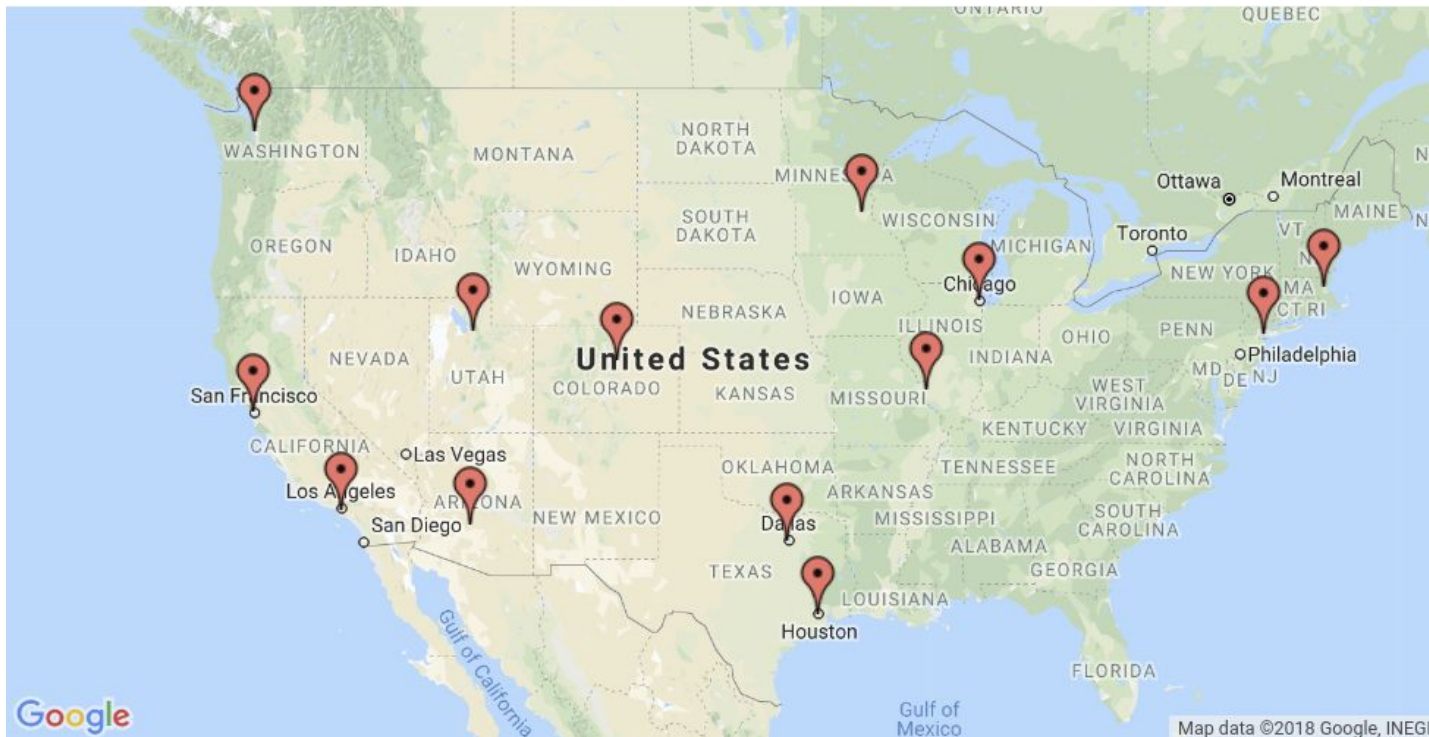
- 장점

- NN보다 더 좋은 해를 찾을 수 있음
 - 경로의 교차를 제거하여 최적화

- 단점

- NN보다 실행 시간이 오래 걸림
 - 여전히 전역 최적해를 보장하지는 않음
 - 초기 해의 품질에 따라 결과가 달라질 수 있음

외판원 순회 문제 (Traveling Salesman Problem, TSP)



외판원 순회 문제 (Traveling Salesman Problem, TSP)

```
(.venv) PS D:\marker-main> python .\17_tsp.py
Nearest Neighbor Distance for 8 cities: 8976.53 km
Path: New York -> Philadelphia -> Chicago -> Houston ->
Time taken: 0.02 milliseconds

Nearest Neighbor Distance for 20 cities: 15701.07 km
Path: New York -> Philadelphia -> Washington DC -> Boston ->
-> Phoenix -> Las Vegas -> Los Angeles -> San Diego ->
Time taken: 0.05 milliseconds

2-opt Distance for 8 cities: 8561.45 km
Path: New York -> Philadelphia -> Houston -> San Antonio
Time taken: 0.14 milliseconds

2-opt Distance for 20 cities: 13557.71 km
Path: New York -> Boston -> Detroit -> Indianapolis ->
s -> Phoenix -> Denver -> Dallas -> San Antonio -> Houston
Time taken: 6.00 milliseconds
```

```
Testing for 8 cities...
Brute Force Distance for 8 cities: 8561.45
Path: Chicago -> New York -> Philadelphia -> Houston
Time taken: 120.80 milliseconds

Testing for 9 cities...
Brute Force Distance for 9 cities: 8855.02
Path: New York -> Philadelphia -> Dallas -> Houston
Time taken: 1217.90 milliseconds

Testing for 10 cities...
Brute Force Distance for 10 cities: 9595.09
Path: New York -> Philadelphia -> Dallas -> Houston
ork
Time taken: 13544.66 milliseconds

Testing for 11 cities...
Brute Force Distance for 11 cities: 10492.32
Path: New York -> Philadelphia -> Dallas -> Houston
go -> New York
Time taken: 174319.44 milliseconds
```

외판원 순회 문제 (Traveling Salesman Problem, TSP)

- 20개 도시에 대한 Brute-Force 예상 실행 시간은?
 - 8개 도시: 0.13초 (8!)
 - 9개 도시: 1.21초 (9!)
 - 10개 도시: 13.85초 (10!)
 - 11개 도시: 168.48초 (11!)
- Brute-Force 알고리즘의 시간 복잡도는 $O(n!)$
- $20! = 2.43 \times 10^{18}$
- $11! = 39,916,800$
- 20!과 11!의 비율을 사용하여 예상 시간을 계산하면:

$$\text{예상 시간} = 168.48\text{초} \times \frac{20!}{11!} \approx 1.02 \times 10^{13}\text{초}$$

- 이는 약 324,000년
 - 우리 은하의 지름은 100,000광년
 - 가장 오래된 호모사피엔스 화석의 나이 300,000년

scikit-learn 라이브러리

- datasets
 - 예제 데이터셋 제공
- preprocessing
 - 데이터 전처리 기능
- model_selection
 - 교차 검증을 위한 데이터셋 분할 도구
 - 모델 평가 도구 제공

scikit-learn 로 TSP 문제의 휴리스틱 해결

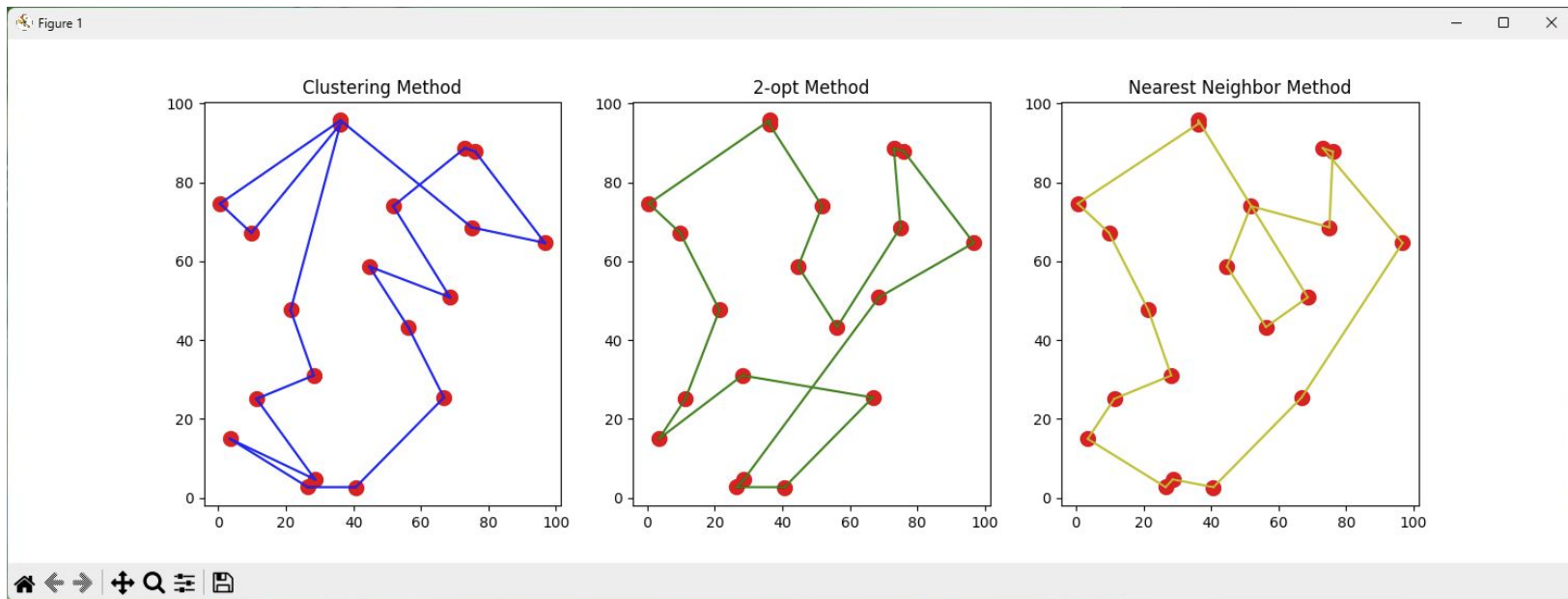
- K-Means 클러스터링
 - 비지도 학습 알고리즘
 - 무작위로 K개의 중심점(centroid) 선택
 - 각 데이터 포인트를 가장 가까운 중심점에 할당
 - 각 클러스터들의 평균을 계산하여 새로운 중심점 설정
 - 중심점이 더 이상 변하지 않을 때까지 반복

scikit-learn 로 TSP 문제의 휴리스틱 해결

- K-Means 클러스터링으로 도시들을 n 개의 그룹으로 묶음
- `pairwise_distances` 전처리기를 사용해 모든 도시 쌍 사이의 거리는 행렬을 생성
- 도시 그룹내에서 경로 최적화
- 도시 그룹간 경로 최적화

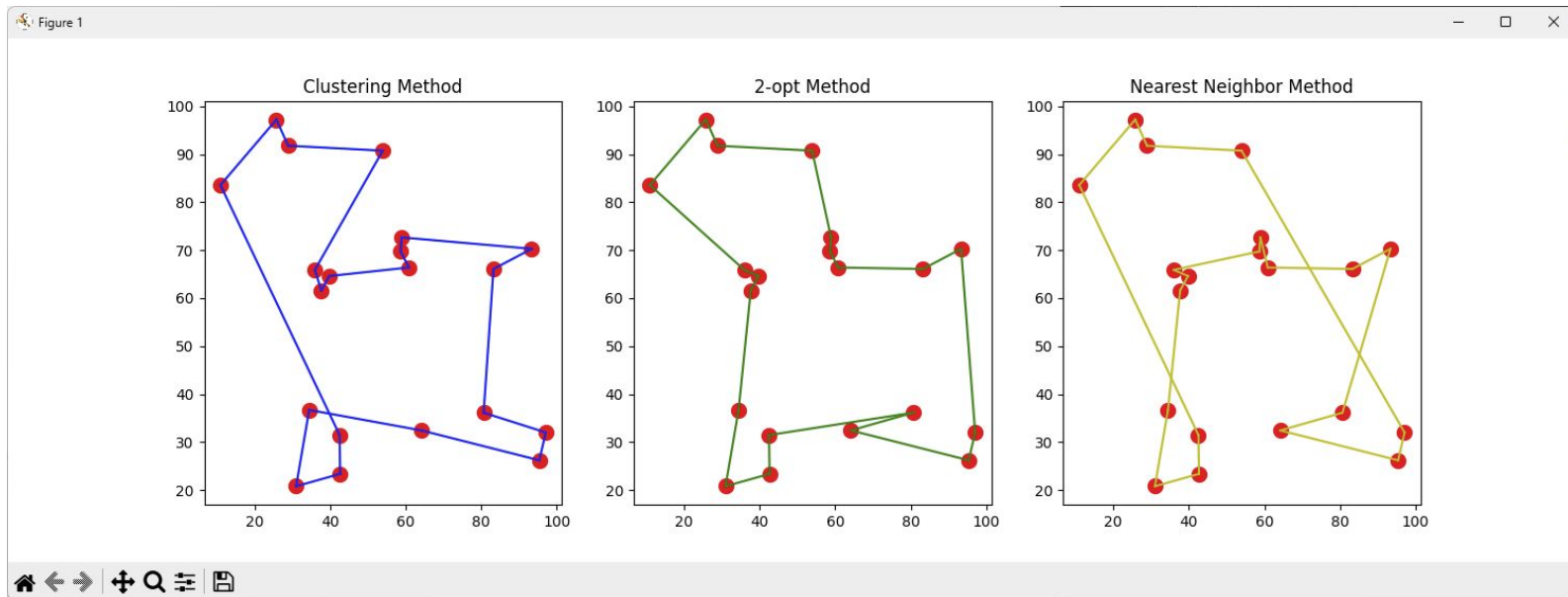
scikit-learn 로 TSP 문제의 휴리스틱 해결

```
(.venv) PS D:\marker-main> python .\18_tsp_scikit-learn.py  
클러스터링 기반 해법의 총 거리: 529.07  
2-opt 해법의 총 거리: 475.32  
Nearest Neighbor 해법의 총 거리: 436.40
```



scikit-learn 로 TSP 문제의 휴리스틱 해결

```
(.venv) PS D:\marker-main> python .\18_tsp_scikit-learn.py  
클러스터링 기반 해법의 총 거리: 376.08  
2-opt 해법의 총 거리: 341.77  
Nearest Neighbor 해법의 총 거리: 411.36
```



알고리즘 학습 추가자료

1. 영리한 프로그래밍을 위한 알고리즘 강좌

<https://www.inflearn.com/course/%EC%95%8C%EA%B3%A0%EB%A6%AC%EC%A6%98-%EA%B0%95%EC%A2%8C>

2. K-MOOC 알고리즘 강좌

<https://www.kmooc.kr/view/course/detail/12400>

3. 어서와! 자료구조와 알고리즘은 처음이지? (유료)

<https://school.programmers.co.kr/learn/courses/57>

알고리즘 학습 추가자료

1. Coursera Stanford University 알고리즘 특화 과정
<https://www.coursera.org/specializations/algorithms>
2. Coursera Princeton University 알고리즘
<https://www.coursera.org/learn/algorithms-part1>
3. MIT Introduction to Algorithms
https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-fall-2011/video_galleries/lecture-videos/

감사합니다